

SISTEMAS DE COMPUTADORES – 2025/2026
Exame Época Normal

Versão A

Apenas autorizada a consulta da folha oficial
Duração: 30m

Nome: _____ Nº _____

Grupo I
(8/20 valores)

NOTAS:

- 1. Em todas as questões deverá assinalar apenas uma resposta.**
- 2. Se a resposta assinalada for incorreta sofrerá uma penalização de 1/3 da cotação da pergunta.**

1 – Considere um processo que possui um *handler* para o sinal `SIGUSR1`. Enquanto esse *handler* está a ser executado, o processo recebe novamente um `SIGUSR1`. Assumindo sinais UNIX standard (*non real-time*), qual dos seguintes comportamentos pode ocorrer?

- a) O sistema operativo cria uma entrada numa fila por cada ocorrência do sinal
- b) O *handler* é imediatamente interrompido e executado novamente de forma recursiva
- c) O sistema operativo regista apenas que existe pelo menos uma nova ocorrência pendente do sinal
- d) O segundo sinal é sempre descartado

2 – Um processo pretende ler todos os dados disponíveis num *pipe* e terminar quando atingir o fim da comunicação (`EOF`). Contudo, por lapso, mantém aberto um descritor de escrita desse mesmo *pipe* que nunca utiliza. Qual poderá ser a consequência?

- a) A função `read()` devolverá imediatamente -1
- b) O *pipe* será automaticamente fechado pelo sistema operativo
- c) O processo poderá ficar bloqueado indefinidamente à espera de novos dados
- d) A função `read()` devolverá imediatamente `EOF`

3 – Um processo cria um *pipe*, armazena os seus descritores em variáveis e posteriormente executa a função `execvp()`. Porque é que o programa carregado pela função `execvp()` não pode aceder diretamente a essas variáveis?

- a) Porque todos os descritores de ficheiro são fechados no início da execução da função `execvp()`
- b) Porque os *pipes* apenas existem durante a execução do programa original
- c) Porque a imagem de memória do processo é substituída e as variáveis deixam de existir
- d) Porque os descritores são renumerados automaticamente

4 – Uma implementação de exclusão mútua baseada apenas em variáveis partilhadas parece funcionar corretamente em ambientes que assumem consistência sequencial, mas falha ocasionalmente num processador moderno *multicore*. Qual é a explicação mais provável?

- a) Os sistemas modernos não suportam memória partilhada
- b) Os compiladores e processadores podem reordenar operações de memória
- c) O algoritmo apenas funciona quando existem mais de dois núcleos
- d) Os sistemas operativos modernos impedem o acesso concorrente a variáveis globais

5 – Considere uma implementação do problema produtor-consumidor com *buffer* limitado utilizando os semáforos *mutex*, *slots*, e *items*. Considere o seguinte código executado por um produtor:

```
/* mutex = 1 */
/* slots = número de posições livres */
sem_wait(mutex);
sem_wait(slots);
```

Qual é o principal problema potencial desta sequência?

- a) Pode ocorrer violação da exclusão mútua
- b) Pode ocorrer inversão de prioridades
- c) Pode ocorrer um impasse (*deadlock*) se o produtor bloquear em *slots* enquanto mantém *mutex* adquirido
- d) Pode ocorrer perda de dados no *buffer* partilhado

6 – Dois processos tentam obter dois recursos utilizando abordagens não bloqueantes (*trylock()*). Sempre que falham, libertam os recursos já adquiridos e repetem imediatamente a tentativa. Ambos os processos permanecem ativos e continuam a executar, mas nenhum consegue completar a sua tarefa. Como se designa esta situação?

- a) Inanição (*Starvation*)
- b) Impasse (*Deadlock*)
- c) Condição de corrida (*Race condition*)
- d) Bloqueio ativo (*Livelock*)

7 – Numa implementação FIFO do problema leitores-escritores, considere que R representa um pedido de acesso por um leitor e W representa um pedido de acesso por um escritor. A fila de espera contém os seguintes pedidos: R1, R2, R3, W1, R4. Assumindo que não existe atualmente nenhum leitor ou escritor a aceder ao recurso partilhado, quais dos pedidos podem avançar imediatamente?

- a) Apenas R1
- b) Apenas R1 e R2
- c) R1, R2 e R3
- d) R1, R2, R3 e R4

8 – Uma aplicação *multithread* apresenta um desempenho muito inferior ao esperado. A análise mostra que cada *thread* atualiza uma variável diferente, nunca acedem às variáveis de outras *threads*, mas todas as variáveis se encontram na mesma linha de cache. Qual o fenómeno mais provável?

- a) Condição de corrida (*Race condition*)
- b) Inversão de prioridades
- c) Falsa partilha (*False sharing*)
- d) Impasse (*Deadlock*)

9 – Qual das seguintes descrições corresponde ao comportamento correto de *pthread_cond_wait()*?

- a) Bloqueia a *thread* mantendo o *mutex* bloqueado durante todo o período de espera
- b) Liberta o *mutex* de forma atômica quando entra em espera e readquire-o antes de regressar da função
- c) Liberta permanentemente o *mutex* antes de bloquear
- d) Apenas bloqueia se não existirem sinais pendentes

10 – Considere o seguinte código:

```
pthread_mutex_lock(&mutex);
while (count == 0)
    pthread_cond_wait(&not_empty, &mutex);
item = buffer_remove();
pthread_mutex_unlock(&mutex);
```

Porque é que a utilização de *if* em vez de *while* neste contexto pode ser incorreta?

- a) Porque *pthread_cond_wait()* pode retornar mesmo que a condição ainda não seja verdadeira
- b) Porque *pthread_cond_wait()* apenas funciona corretamente dentro de ciclos
- c) Porque o compilador pode reordenar incorretamente a instrução *if*
- d) Porque o *mutex* pode ser destruído durante a espera

SISTEMAS DE COMPUTADORES – 2025/2026
Exame Época Normal

Versão A

Apenas autorizada a consulta da folha oficial
Duração: 1h 30m

Grupo II
(12/20 valores)

NOTAS:

1. Responder cada questão numa folha separada, devidamente identificada.
2. Indicar a versão do exame em cada uma das folhas.
3. Não é necessário indicar o nome das bibliotecas usadas na resolução.

[5 valores] 1) A equipa na qual se encontra integrado tem como objetivo desenvolver um programa em C que utilize processos e *pipes* para calcular o maior valor de um conjunto de números inteiros. O programa recebe pela linha de comandos uma sequência de números inteiros positivos. Pode assumir que existem sempre pelo menos 2 números e o número de argumentos é sempre par.

O processo pai deve:

- criar dois processos filho;
- dividir o vetor de números em duas metades;
- enviar cada metade para um dos filhos através de um *pipe*;
- receber de cada filho o maior valor calculado e determinar e imprimir o maior valor global.

Cada processo filho deve:

- receber de um *pipe* o conjunto de números enviado pelo pai através do *pipe* correspondente;
- calcular o maior valor recebido;
- enviar o resultado de volta ao pai através de um *pipe*.

Assuma que os processos filho **já foram criados** e os arrays `int parent_to_child[2][2]` e `child_to_parent[2][2]` foram corretamente inicializados com a função `pipe()`, mas que **todos os descritores ainda se encontram abertos**.

Foram-lhe atribuídas as tarefas de desenvolvimento das funções responsáveis pela troca de valores entre os processos e cálculo do maior valor global, pelo que deverá implementar apenas estas:

- `int parent_function(int parent_to_child[2][2],
child_to_parent[2][2], int *vec, int n);`
- `void child_function(int child_id, int parent_to_child[2][2],
child_to_parent[2][2]);`

[7 valores] 2) A equipa na qual se encontra integrado tem como objetivo desenvolver um programa em C que simule uma fila de impressão partilhada. Existem várias *threads* produtoras, representando utilizadores que submetem trabalhos de impressão, e várias *threads* consumidoras, representando impressoras. Cada trabalho de impressão é representado por um número inteiro correspondente ao seu identificador.

Os produtores devem:

- inserir trabalhos numa fila circular com capacidade limitada;
- caso a fila esteja cheia, o **produtor deve bloquear** até existir espaço disponível.

As impressoras devem:

- remover trabalhos da fila;
- caso a fila esteja vazia, a **impressora deve bloquear** até existir pelo menos um trabalho disponível.

A sincronização deve ser realizada **exclusivamente** com *mutexes* e variáveis de condição (não deve usar espera ativa (*busy waiting*)). Assuma que a estrutura de dados da fila já foi implementada e inicializada corretamente:

```
#define BUFFER_SIZE 10

typedef struct {
    int jobs[BUFFER_SIZE];
    int head;
    int tail;
    int count;
    pthread_mutex_t mutex;
    pthread_cond_t not_full;
    pthread_cond_t not_empty;
} print_queue_t;
```

Assuma ainda que:

- os *mutexes* e variáveis de condição já foram inicializados;
- a fila se encontra inicialmente vazia;
- existe apenas uma fila partilhada por todas as *threads*.

Foram-lhe atribuídas as tarefas de desenvolvimento das funções responsáveis pela inserção e remoção de trabalhos, pelo que deverá implementar apenas estas:

- `void submit_job(print_queue_t *queue, int job_id);`
- `int get_job(print_queue_t *queue);`